

OS SKILL 12

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

#define MAX_COMMANDS 10

/*
 * Executes a pipeline containing multiple commands.
 *
 * Example:
 *   ls -l | grep ".c" | wc -l
 */
void execute_pipeline(char ***commands, int command_count)
{
    int pipe_count = command_count - 1;

    /* Descriptor array for multiple pipes */
    int pipes[MAX_COMMANDS][2];

    pid_t pids[MAX_COMMANDS];

    printf("\n=====\\n");
    printf("Creating pipeline with %d commands\\n", command_count);
    printf("Number of pipes required: %d\\n", pipe_count);
    printf("=====\\n");

    /* Create all required pipes */
    for (int i = 0; i < pipe_count; i++) {
        if (pipe(pipes[i]) == -1) {
            perror("pipe");
            exit(EXIT_FAILURE);
        }

        printf("Pipe %d created: read=%d write=%d\\n",
            i,
            pipes[i][0],
            pipes[i][1]);
    }
}
```

```

fflush(stdout);

/* Create one child process for every command */
for (int i = 0; i < command_count; i++) {

    pids[i] = fork();

    if (pids[i] < 0) {
        perror("fork");
        exit(EXIT_FAILURE);
    }

    if (pids[i] == 0) {

        /*
         * If this is not the first command,
         * input should come from previous pipe.
         */
        if (i > 0) {

            if (dup2(pipes[i - 1][0], STDIN_FILENO) == -1) {
                perror("dup2 input");
                exit(EXIT_FAILURE);
            }
        }

        /*
         * If this is not the last command,
         * output should go to next pipe.
         */
        if (i < command_count - 1) {

            if (dup2(pipes[i][1], STDOUT_FILENO) == -1) {
                perror("dup2 output");
                exit(EXIT_FAILURE);
            }
        }

        /*
         * Child no longer needs any original
         * pipe descriptors after dup2().
         */
        for (int j = 0; j < pipe_count; j++) {
            close(pipes[j][0]);
            close(pipes[j][1]);
        }
    }
}

```

```

    }

    /* Execute the command */
    execvp(commands[i][0], commands[i]);

    /* Only reached when execvp fails */
    perror("execvp");
    exit(EXIT_FAILURE);
}

printf("Started child %d with PID %d for command: %s\n",
       i + 1,
       pids[i],
       commands[i][0]);
}

/*
 * Parent does not use pipe descriptors,
 * so close all of them.
 */
printf("\nParent closing unused pipe descriptors...\n");

for (int i = 0; i < pipe_count; i++) {
    close(pipes[i][0]);
    close(pipes[i][1]);
}

/*
 * Synchronize processes.
 */
printf("Parent waiting for child processes...\n\n");

for (int i = 0; i < command_count; i++) {
    int status;

    waitpid(pids[i], &status, 0);

    if (WIFEXITED(status)) {
        printf("Child PID %d exited with status %d\n",
               pids[i],
               WEXITSTATUS(status));
    }
    else {

```

```

        printf("Child PID %d terminated abnormally\n",
               pids[i]);
    }
}

printf("\nAll child processes completed.\n");
printf("Pipeline resources cleaned successfully.\n");
}

/* Display pipeline structure */
void display_pipeline(char ***commands, int count)
{
    printf("\nPipeline Layout:\n");

    for (int i = 0; i < count; i++) {
        for (int j = 0; commands[i][j] != NULL; j++) {
            printf("%s", commands[i][j]);

            if (commands[i][j + 1] != NULL)
                printf(" ");
        }

        if (i < count - 1)
            printf(" | ");
    }

    printf("\n");
}

int main()
{
    printf("=====\n");
    printf("      TASK 12 - PIPE MANAGEMENT      \n");
    printf("=====\n");

    /*
     * TEST 1
     *
     * printf ... | grep apple | wc -l
     *
     * Tests:
    */
}

```

```

/*
 * - pipe creation
 * - stream redirection
 * - connected commands
 * - process synchronization
 */

printf("\n\n=== TEST 1: THREE COMMAND PIPELINE ===\n");

char *cmd1[] = {
    "printf",
    "apple\nbanana\napple\norange\n",
    NULL
};

char *cmd2[] = {
    "grep",
    "apple",
    NULL
};

char *cmd3[] = {
    "wc",
    "-l",
    NULL
};

char **pipeline1[] = {
    cmd1,
    cmd2,
    cmd3
};

display_pipeline(pipeline1, 3);

printf("\nExpected data flow:\n");
printf("printf -> grep apple -> wc -l\n");
printf("Expected result: 2\n");

execute_pipeline(pipeline1, 3);

/*
 * TEST 2
 *
 * seq 1 20 | grep 1 | sort -n | wc -l
 */

```

```
printf("\n\n=== TEST 2: FOUR COMMAND PIPELINE ===\n");
```

```
char *cmd4[] = {  
    "seq",  
    "1",  
    "20",  
    NULL  
};
```

```
char *cmd5[] = {  
    "grep",  
    "1",  
    NULL  
};
```

```
char *cmd6[] = {  
    "sort",  
    "-n",  
    NULL  
};
```

```
char *cmd7[] = {  
    "wc",  
    "-l",  
    NULL  
};
```

```
char **pipeline2[] = {  
    cmd4,  
    cmd5,  
    cmd6,  
    cmd7  
};
```

```
display_pipeline(pipeline2, 4);
```

```
printf("\nTesting long pipeline execution...\n");
```

```
execute_pipeline(pipeline2, 4);
```

```
/*  
 * TEST 3  
 *  
 * ls -l | wc -l  
 */
```

```

printf("\n\n=== TEST 3: TWO COMMAND PIPELINE ===\n");

char *cmd8[] = {
    "ls",
    "-l",
    NULL
};

char *cmd9[] = {
    "wc",
    "-l",
    NULL
};

char **pipeline3[] = {
    cmd8,
    cmd9
};

display_pipeline(pipeline3, 2);

execute_pipeline(pipeline3, 2);

printf("\n===== \n");
printf("          DATA FLOW TEST COMPLETE          \n");
printf("===== \n");

printf("\nTask verified successfully:\n");
printf("- Pipes created\n");
printf("- Streams redirected using dup2()\n");
printf("- Commands connected\n");
printf("- File descriptors closed\n");
printf("- Processes synchronized using waitpid()\n");
printf("- Multiple pipes managed\n");
printf("- Descriptor arrays used\n");
printf("- Multiple processes launched\n");
printf("- Resources cleaned\n");
printf("- Long pipelines tested\n");

printf("\n=== TASK 12 COMPLETE ===\n");

return 0;
}

```

```
=====
TASK 12 - PIPE MANAGEMENT
=====
```

```
=== TEST 1: THREE COMMAND PIPELINE ===
```

```
Pipeline Layout:
```

```
printf apple
```

```
banana
```

```
apple
```

```
orange
```

```
| grep apple | wc -l
```

```
Expected data flow:
```

```
printf -> grep apple -> wc -l
```

```
Expected result: 2
```

Expected Result: 2

```
=====
Creating pipeline with 3 commands
Number of pipes required: 2
=====
Pipe 0 created: read=3 write=4
Pipe 1 created: read=5 write=6
Started child 1 with PID 1163 for command: printf
Started child 2 with PID 1164 for command: grep
Started child 3 with PID 1165 for command: wc

Parent closing unused pipe descriptors...
Parent waiting for child processes...

Child PID 1163 exited with status 0
2
Child PID 1164 exited with status 0
Child PID 1165 exited with status 0

All child processes completed.
Pipeline resources cleaned successfully.

=== TEST 2: FOUR COMMAND PIPELINE ===

Pipeline Layout:
seq 1 20 | grep 1 | sort -n | wc -l

Testing long pipeline execution...
```

```
=====
Creating pipeline with 4 commands
Number of pipes required: 3
=====
Pipe 0 created: read=3 write=4
Pipe 1 created: read=5 write=6
Pipe 2 created: read=7 write=8
Started child 1 with PID 1166 for command: seq
Started child 2 with PID 1167 for command: grep
Started child 3 with PID 1168 for command: sort
Started child 4 with PID 1169 for command: wc

Parent closing unused pipe descriptors...
Parent waiting for child processes...

Child PID 1166 exited with status 0
Child PID 1167 exited with status 0
11
Child PID 1168 exited with status 0
Child PID 1169 exited with status 0

All child processes completed.
Pipeline resources cleaned successfully.

=== TEST 3: TWO COMMAND PIPELINE ===

Pipeline Layout:
ls -l | wc -l
```

```
=====
Creating pipeline with 2 commands
Number of pipes required: 1
=====
Pipe 0 created: read=3 write=4
Started child 1 with PID 1171 for command: ls
Started child 2 with PID 1172 for command: wc

Parent closing unused pipe descriptors...
Parent waiting for child processes...

2
Child PID 1171 exited with status 0
Child PID 1172 exited with status 0

All child processes completed.
Pipeline resources cleaned successfully.

=====
DATA FLOW TEST COMPLETE
=====

Task verified successfully:
- Pipes created
- Streams redirected using dup2()
- Commands connected
- File descriptors closed
- Processes synchronized using waitpid()
- Multiple pipes managed
- Descriptor arrays used
- Multiple processes launched
- Resources cleaned
- Long pipelines tested

=== TASK 12 COMPLETE ===
```